Visual Studio (Windows)

This page covers the full workflow for working on SWAT+ in Visual Studio 2022 on Windows: installing the required tools, cloning the repository, configuring the CMake build, and running a debug session against an input dataset.

Screenshots referenced below live in the SWAT+ source repo's `doc/` folder.

Required software

Install in this order. The Intel compiler requires Visual Studio to be present first.

1. Visual Studio Community 2022. During install, in the **Workloads** panel, check **Desktop development with C++**. That workload pulls in the CMake integration that SWAT+ needs.
2. Intel Fortran Compiler for Windows. Use the standalone installer. The current Intel oneAPI release ships `ifx`. The older `ifort` was deprecated in 2024 and is gone from the 2025 release. SWAT+ produces the same results with either Intel compiler.
3. Git for Windows. Visual Studio bundles its own Git, but SWAT+ uses the standalone `git` during the CMake build to read the current tag for the version string.

Reboot after the three installs finish.

Optional: install Python (from the Microsoft Store) and the Python workload in Visual Studio if you want to run the integrated test suite from inside the IDE.

Starting Visual Studio

The Intel compiler only works with the Visual Studio CMake integration if Visual Studio is launched from the Intel oneAPI command prompt. Follow these steps every time.

1. Open the Windows Start Menu, type `intel`, and select **Intel oneAPI command prompt for Intel 64 for Visual Studio 2022**. Pin it if you use it often.
2. The terminal opens and prints the oneAPI initialization output:

   ```
   :: initializing oneAPI environment...
      Initializing Visual Studio command-line environment...
      Visual Studio version 17.13.0 environment configured.
      "C:\Program Files\Microsoft Visual Studio\2022\Community\"
      Visual Studio command-line environment initialized for: 'x64'
   :  compiler -- latest
   :  debugger -- latest
   :: oneAPI environment initialized ::
   ```

3. At the prompt, run:

   ```
   devenv
   ```

   This launches Visual Studio with the Intel environment variables set.

If you start Visual Studio by clicking the desktop icon directly, the Intel compiler will not be found and the build will fail.

Cloning the repository

Visual Studio can clone the SWAT+ repository and open it as a CMake project in one step.

1. On the Visual Studio start screen, click **Clone a Repository**.
2. Click the GitHub icon (lower left) to browse your repositories. Sign in if prompted. Select your fork of `swatplus` and click **Open**. Adjust the local path if needed.
3. Visual Studio clones the repository and loads it into Solution Explorer as a CMake project. It runs the CMake configure step against whichever Fortran compiler it finds (`ifx`, `ifort`, or `gfortran`). The 2025 Intel compiler only ships `ifx`, so any `ifort` configurations will fail on that version.

Configuring, compiling, running

Configure

SWAT+ is a CMake project. The Visual Studio toolbar exposes the available compiler and build-type combinations through a drop-down. Switching the selection triggers a reconfigure. The Output window prints the CMake settings used, which is the place to verify which compiler picked up.

To force a fresh configure, right-click `CMakeLists.txt` in Solution Explorer and select **Delete Cache and Reconfigure**.

Build

Use **Build > Build All** or **Build > Rebuild All**. The executable name encodes the compiler, architecture, and version. The version is read from the latest Git tag automatically. See Tagging and releases.

The Intel-built Windows executable is dynamically linked. To run it on a machine without Visual Studio, ship the Intel redistributable libraries alongside it.

Run and debug

You need to tell the project which folder contains the input dataset to run against.

1. Right-click `CMakeLists.txt` in Solution Explorer and select **Add Debug Configuration**. If Co-Pilot is hiding Solution Explorer, close that pane first.
2. In the dialog, pick **Default** and click **Select**. Visual Studio creates `launch.vs.json` if it does not exist and opens it for editing.

The default file looks like this:

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "CMakeLists.txt"
    }
  ]
}
```

Add a `currentDir` entry pointing at your input dataset. The path can be absolute or relative to the `swatplus` project root. The `name` field is what shows up in the toolbar.

```
{
  "version": "0.2.1",
  "defaults": {},
  "configurations": [
    {
```

```
      "type": "default",
      "project": "CMakeLists.txt",
      "projectTarget": "",
      "name": "Ames_testing",
      "currentDir": "C:/work/sp/Ames_testing"
    }
  ]
}
```

Do not change `project` or `type`. You can add more configurations by repeating **Add Debug Configuration** and editing the new block.

Pick your configuration from the toolbar drop-down and run **Debug > Start Debugging** or **Debug > Start Without Debugging**. Breakpoints, conditional watches, and step-through all work.

Resources

- Intel C++ and Fortran Compilers Redistributable Libraries by Version
- Intel Fortran Compiler
- Intel Fortran Compiler for oneAPI release notes
- Visual Studio documentation
- CMake